Kernel Density Estimation (KDE)

Intuition: How KDE Works

- Imagine we have N data points on the number line:

$$x_1, x_2, x_3, \ldots, x_N.$$

- Kernel Density Estimation (KDE) builds a **smooth pdf** from data.
- Instead of plotting them as spikes (like a histogram), we **smooth them out**.
- For each point x_j:
  - Place a small smooth curve (kernel bump) centered at x_j.
  - The bandwidth h_j decides how wide the bump is.
- Finally, we **add up all bumps**:
  - Where bumps overlap, the curve is tall.
  - Where there are fewer bumps, the curve is low.

This sum of bumps is the Kernel Density Estimate (KDE).

Kernel and Bandwidth

- Kernel $\kappa(u)$: symmetric pdf (e.g. Gaussian).
- Bandwidth $h_j > 0$: controls bump width.
  - Small h_j: narrow and spiky.
  - Large h_j: wide and flat.

$$\widehat{f}(x; \mathbf{h}) = \frac{1}{N} \sum_{j=1}^{N} \frac{1}{h_j} \kappa\Big(\frac{x - x_j}{h_j}\Big).$$

From Kernel PDF to KDE CDF

- Recall the variable-bandwidth KDE (pdf):

$$\widehat{f}(x;\mathbf{h}) = \frac{1}{N}\sum_{j=1}^{N}\frac{1}{h_j}\,\kappa\Big(\frac{x-x_j}{h_j}\Big).$$

- The CDF at point z is the integral of the pdf from $-\infty$ to z:

$$\widehat{F}(z;\mathbf{h}) \;=\; \int_{-\infty}^{z}\widehat{f}(t;\mathbf{h})\,dt \;=\; \int_{-\infty}^{z}\frac{1}{N}\sum_{j=1}^{N}\frac{1}{h_j}\,\kappa\Big(\frac{t-x_j}{h_j}\Big)\,dt.$$

- Swap sum and integral (finite sum — allowed):

$$\widehat{F}(z;\mathbf{h}) \;=\; \frac{1}{N}\sum_{j=1}^{N}\int_{-\infty}^{z}\frac{1}{h_j}\,\kappa\Big(\frac{t-x_j}{h_j}\Big)\,dt.$$

From Kernel PDF to KDE CDF

- Swap sum and integral (finite sum — allowed):

$$\widehat{F}(z;\mathbf{h}) \;=\; \frac{1}{N}\sum_{j=1}^{N}\int_{-\infty}^{z}\frac{1}{h_j}\,\kappa\Big(\frac{t-x_j}{h_j}\Big)\,dt.$$

- For each term do a change of variables:

$$u=\frac{t-x_j}{h_j} \quad\Rightarrow\quad t=x_j+h_j u, \qquad dt=h_j\,du.$$

Under this substitution the integration limits transform as:

$$t=-\infty\mapsto u=-\infty, \qquad t=z\mapsto u=\frac{z-x_j}{h_j}.$$

- Substitute into each integral:

$$\int_{-\infty}^{z}\frac{1}{h_j}\,\kappa\Big(\frac{t-x_j}{h_j}\Big)\,dt=\int_{-\infty}^{\frac{z-x_j}{h_j}}\frac{1}{h_j}\,\kappa(u)\,(h_j\,du)=\int_{-\infty}^{\frac{z-x_j}{h_j}}\kappa(u)\,du.$$

Notice the h_j factors cancel.

From Kernel PDF to KDE CDF

- The remaining integral is exactly the kernel CDF evaluated at $\dfrac{z - x_j}{h_j}$:

$$\int_{-\infty}^{\frac{z-x_j}{h_j}} \kappa(u)\, du = K\Big(\frac{z - x_j}{h_j}\Big).$$

- Summing over j gives the final KDE-CDF formula:

$$\boxed{\widehat{F}(z; \mathbf{h}) \;=\; \frac{1}{N} \sum_{j=1}^{N} K\Big(\frac{z - x_j}{h_j}\Big).}$$

Matrix Form

- Evaluate CDF at points $z_1, \ldots, z_M$.
- Define the matrix:

$$\left[K_{\mathrm{mat}}(\mathbf{h})\right]_{k,j} = K\left(\frac{z_k - x_j}{h_j}\right).$$

- Then

$$\widehat{\mathbf{F}}(\mathbf{h}) = \frac{1}{N} K_{\mathrm{mat}}(\mathbf{h})\, \mathbf{1}_N.$$

Matching the Empirical CDF

- We do not compare CDFs at all real values, but only at a set of **evaluation points**.
- A natural choice: use the **midpoints between the sorted data points**

$$z_k = \frac{x_k + x_{k+1}}{2}, \qquad k = 1, \ldots, M, \quad M = N - 1.$$

- Empirical CDF targets at these midpoints are

$$T_k = \frac{k}{N}, \qquad k = 1, \ldots, M.$$

- Collect as a vector:

$$\mathbf{T} = \left(\tfrac{1}{N}, \tfrac{2}{N}, \ldots, \tfrac{M}{N}\right)^\top.$$

- Loss function (squared error):

$$J(\mathbf{h}) = \|\widehat{\mathbf{F}}(\mathbf{h}) - \mathbf{T}\|_2^2.$$

Gradient Derivation: Step by Step

- Recall the loss function:

$$J(\mathbf{h}) = \|\widehat{\mathbf{F}}(\mathbf{h}) - \mathbf{T}\|_2^2.$$

  Define the residual:

$$\mathbf{r}(\mathbf{h}) = \widehat{\mathbf{F}}(\mathbf{h}) - \mathbf{T}.$$

- Expanding:

$$J(\mathbf{h}) = \mathbf{r}(\mathbf{h})^\top \mathbf{r}(\mathbf{h}).$$

- Differentiate w.r.t. h_j using the chain rule:

$$\frac{\partial J}{\partial h_j} = 2\,\mathbf{r}(\mathbf{h})^\top \frac{\partial \mathbf{r}(\mathbf{h})}{\partial h_j}.$$

- But $\mathbf{r}(\mathbf{h}) = \widehat{\mathbf{F}}(\mathbf{h}) - \mathbf{T}$, and $\mathbf{T}$ is constant. So:

$$\frac{\partial J}{\partial h_j} = 2\,\mathbf{r}(\mathbf{h})^\top \frac{\partial \widehat{\mathbf{F}}(\mathbf{h})}{\partial h_j}.$$

Gradient Derivation: Step by Step

- Recall the CDF vector:

$$\widehat{\mathbf{F}}(\mathbf{h}) = \frac{1}{N} K_{\mathrm{mat}}(\mathbf{h})\, \mathbf{1}_N.$$

- Differentiate this w.r.t. h_j:

$$\frac{\partial \widehat{\mathbf{F}}(\mathbf{h})}{\partial h_j} = \frac{1}{N} \frac{\partial K_{\mathrm{mat}}(\mathbf{h})}{\partial h_j}\, \mathbf{1}_N.$$

- Substitute back into the gradient formula:

$$\boxed{\frac{\partial J}{\partial h_j} = \frac{2}{N}\, \mathbf{r}(\mathbf{h})^\top \Big(\frac{\partial K_{\mathrm{mat}}(\mathbf{h})}{\partial h_j}\, \mathbf{1}_N \Big).}$$

Derivative of Entries (Gaussian Kernel)

- Focus on a single matrix entry:

$$\left[K_{\mathrm{mat}}(\mathbf{h})\right]_{k,j} = K\Big(\frac{z_k - x_j}{h_j}\Big).$$

- Differentiate w.r.t. h_j (other h_ℓ, $\ell \neq j$, have no effect):

$$\frac{\partial}{\partial h_j} K\Big(\frac{z_k - x_j}{h_j}\Big).$$

- Apply the chain rule: derivative of $K(\cdot)$ is $\kappa(\cdot)$, the kernel pdf.

$$= K'\Big(\frac{z_k - x_j}{h_j}\Big) \cdot \frac{\partial}{\partial h_j}\Big(\frac{z_k - x_j}{h_j}\Big).$$

- Compute the inner derivative:

$$\frac{\partial}{\partial h_j}\Big(\frac{z_k - x_j}{h_j}\Big) = -\frac{z_k - x_j}{h_j^2}.$$

Derivative of Entries (Gaussian Kernel)

- Substitute back:

$$\frac{\partial}{\partial h_j} K\Big(\frac{z_k - x_j}{h_j}\Big) = \kappa\Big(\frac{z_k - x_j}{h_j}\Big)\Big(-\frac{z_k - x_j}{h_j^2}\Big).$$

- Final simplified form:

$$\boxed{\frac{\partial}{\partial h_j} K\Big(\frac{z_k - x_j}{h_j}\Big) = -\kappa\Big(\frac{z_k - x_j}{h_j}\Big)\frac{z_k - x_j}{h_j^2}.}$$

Structure of $\dfrac{\partial K_{\text{mat}}}{\partial h_j}$

Definition.

$$\left(K_{\text{mat}}(\mathbf{h})\right)_{k,\ell} = K\left(\frac{z_k - x_\ell}{h_\ell}\right), \qquad K_{\text{mat}}(\mathbf{h}) \in \mathbb{R}^{M\times N}.$$

Differentiate w.r.t. h_j**:**

$$\frac{\partial}{\partial h_j}\left(K_{\text{mat}}(\mathbf{h})\right)_{k,\ell} = \begin{cases} 0, & \ell \neq j, \\ -\kappa\left(\frac{z_k - x_j}{h_j}\right)\dfrac{z_k - x_j}{h_j^2}, & \ell = j. \end{cases}$$

Matrix structure. Only the j-th column is nonzero:

$$\frac{\partial K_{\text{mat}}}{\partial h_j} = \begin{bmatrix} 0 & \cdots & -\kappa\left(\frac{z_1 - x_j}{h_j}\right)\frac{z_1 - x_j}{h_j^2} & \cdots & 0 \\ 0 & \cdots & -\kappa\left(\frac{z_2 - x_j}{h_j}\right)\frac{z_2 - x_j}{h_j^2} & \cdots & 0 \\ \vdots & & \vdots & & \vdots \\ 0 & \cdots & -\kappa\left(\frac{z_M - x_j}{h_j}\right)\frac{z_M - x_j}{h_j^2} & \cdots & 0 \end{bmatrix}.$$

Substitute Entrywise Derivative into the Gradient

Recall:

$$\frac{\partial J}{\partial h_j} = \frac{2}{N}\,\mathbf{r}(\mathbf{h})^\top \Big(\frac{\partial K_{\mathrm{mat}}(\mathbf{h})}{\partial h_j}\,\mathbf{1}_N\Big), \qquad \mathbf{r}(\mathbf{h}) = \widehat{\mathbf{F}}(\mathbf{h}) - \mathbf{T}.$$

Entrywise derivative (from previous slide). For each row k of column j,

$$\frac{\partial}{\partial h_j} K\Big(\frac{z_k - x_j}{h_j}\Big) = -\kappa\Big(\frac{z_k - x_j}{h_j}\Big)\,\frac{z_k - x_j}{h_j^2}.$$

Form the vector $\dfrac{\partial K_{\mathrm{mat}}}{\partial h_j}\mathbf{1}_N$:

$$\frac{\partial K_{\mathrm{mat}}}{\partial h_j}\,\mathbf{1}_N = \begin{bmatrix} -\kappa\big(\frac{z_1 - x_j}{h_j}\big)\,\dfrac{z_1 - x_j}{h_j^2} \\ -\kappa\big(\frac{z_2 - x_j}{h_j}\big)\,\dfrac{z_2 - x_j}{h_j^2} \\ \vdots \\ -\kappa\big(\frac{z_M - x_j}{h_j}\big)\,\dfrac{z_M - x_j}{h_j^2} \end{bmatrix} \in \mathbb{R}^M.$$

Substitute Entrywise Derivative into the Gradient

Now take the inner product with the residual:

$$\mathbf{r}(\mathbf{h})^\top \left(\frac{\partial K_{\mathrm{mat}}}{\partial h_j} \mathbf{1}_N \right) = -\sum_{k=1}^{M} r_k(\mathbf{h})\, \kappa\!\left(\frac{z_k - x_j}{h_j} \right) \frac{z_k - x_j}{h_j^2}.$$

Multiply by the factor $\frac{2}{N}$ to obtain the component form:

$$\boxed{\frac{\partial J}{\partial h_j} = -\frac{2}{N} \sum_{k=1}^{M} r_k(\mathbf{h})\, \kappa\!\left(\frac{z_k - x_j}{h_j} \right) \frac{z_k - x_j}{h_j^2}.}$$

Matrix-vector compact form. Define the $M \times N$ matrix $D(\mathbf{h})$ with entries

$$D_{k,j}(\mathbf{h}) = -\kappa\!\left(\frac{z_k - x_j}{h_j} \right) \frac{z_k - x_j}{h_j^2}.$$

Then

$$\boxed{\nabla_{\mathbf{h}} J(\mathbf{h}) = \frac{2}{N} D(\mathbf{h})^\top \mathbf{r}(\mathbf{h}).}$$

Gradient Descent Update

- Update rule:

$$\mathbf{h}^{(t+1)} = \mathbf{h}^{(t)} - \eta \, \nabla_{\mathbf{h}} J(\mathbf{h}^{(t)}).$$

- To keep $h_j > 0$, use log-variables:

$$\alpha_j = \log h_j, \qquad \alpha_j^{(t+1)} = \alpha_j^{(t)} - \eta \, \frac{\partial J}{\partial \alpha_j}.$$

Computations per Gradient Step

At each iteration, two matrix–vector products are required:

1. **Residual evaluation:**

$$\widehat{\mathbf{F}}(\mathbf{h}) = \frac{1}{N} K_{\mathrm{mat}}(\mathbf{h})\, \mathbf{1}_N, \qquad r(\mathbf{h}) = \widehat{\mathbf{F}}(\mathbf{h}) - T.$$

2. **Gradient evaluation:**

$$\nabla_{\mathbf{h}} J(\mathbf{h}) = \frac{2}{N}\, D(\mathbf{h})^\top r(\mathbf{h}).$$

Computations per Gradient Step

At each iteration, two matrix–vector products are required:

1. **Residual evaluation:**

$$\widehat{\mathbf{F}}(\mathbf{h}) = \frac{1}{N} K_{\mathrm{mat}}(\mathbf{h})\, \mathbf{1}_N, \qquad r(\mathbf{h}) = \widehat{\mathbf{F}}(\mathbf{h}) - T.$$

2. **Gradient evaluation:**

$$\nabla_{\mathbf{h}} J(\mathbf{h}) = \frac{2}{N}\, D(\mathbf{h})^\top r(\mathbf{h}).$$

Summary: In each gradient step, the following matrix-vector products need to be evaluated

$$\underbrace{K_{\mathrm{mat}}(\mathbf{h})\, \mathbf{1}_N}_{M \times N} \quad , \quad \underbrace{D(\mathbf{h})^\top r(\mathbf{h})}_{N \times M}.$$

Efficiency depends on exploiting kernel structure (e.g. low-rank/H-matrix methods).

Mat–Vec in Residual Evaluation

$$\widehat{\mathbf{F}}(\mathbf{h}) = \frac{1}{N} K_{\mathrm{mat}}(\mathbf{h})\, \mathbf{1}_N,$$

where

$$K_{k,j}(\mathbf{h}) = K\left(\frac{z_k - x_j}{h_j}\right) \quad \text{(CDF of kernel)}$$

- This is a **kernel interaction matrix** mat–vec.
- Direct cost: $\mathcal{O}(MN)$.
- For smooth kernels, far-field blocks are numerically low-rank.
- $\mathcal{H}$-matrix approximation reduces cost to

$$\mathcal{O}\big(r(M+N)\log N\big), \qquad r \ll \min(M, N).$$

Conclusion: Residual evaluation can be accelerated by $\mathcal{H}$-matrices.

Mat–Vec in Gradient Evaluation

$$\nabla_{\mathbf{h}} J(\mathbf{h}) = \frac{2}{N} D(\mathbf{h})^\top \mathbf{r}(\mathbf{h}), \qquad D_{k,j}(\mathbf{h}) = -\kappa\left(\frac{z_k - x_j}{h_j}\right) \frac{z_k - x_j}{h_j^2}.$$

- Structure is again kernel-like, with an extra polynomial prefactor.
- Direct cost: $\mathcal{O}(MN)$.
- Prefactor does not destroy low-rankness of far-field blocks.
- $\mathcal{H}$-matrix compression again yields nearly linear cost.

Conclusion: Gradient evaluation mat–vecs can also be accelerated by $\mathcal{H}$-matrices.

Key Takeaways

- KDE builds a smooth pdf from samples.
- Bandwidth controls smoothness of bumps.
- Matching KDE-CDF with empirical CDF gives a way to **optimize bandwidths**.
- Gradient descent with H-matrix approximation provides a practical algorithm.